



# AI that **watches every log**, so engineers don't have to.

Proactive, AI-powered monitoring & instant Q&A across our fleet of 90+ microservices — built read-only, cost-aware, and customer-safe by design.

Senior Leadership Briefing

Engineering & Reliability

Townhall Edition



## ● THE PROBLEM

# At 90+ services, logs outpace humans.

We find out about incidents from **customers**, not dashboards. Engineers spend hours grepping logs across services to answer one question.



### Reactive firefighting

Issues surface after customer impact. Mean-time-to-detect is measured in the wrong direction.



### Tribal knowledge

"Is payment-service healthy?" takes a senior engineer, three consoles, and ten minutes.



### Runaway log cost

Storing and scanning everything at 90 services is expensive and only grows.

# 90+

Java microservices in scope

# Hours

Lost per incident to manual log triage

# 24x7

Alert fatigue & on-call toil

- THE VISION

# From **reactive** to **proactive** — with AI on watch.

- **An AI agent inspects the fleet every few minutes** and raises a single, smart alert only when something genuinely breaches our thresholds.
- **Anyone can just ask, in Google Chat:** "Is checkout slow right now?" — and get an answer backed by real logs in seconds.
- **Tuning needs zero code changes** — sensitivity lives in a config file owned by the teams.



## The promise

Catch problems **before customers do**, answer questions **in seconds**, and do it all **without touching production**.

## • WHAT IS LOGSENTRY

# One system. Three capabilities.

A thin, intelligent layer on top of the Google Cloud Logging we *already* have — no new agents on services, no production changes.



### MCP Tool Server

5 read-only tools that query logs, score health, and detect anomalies — the senses of the system.



### AI Monitoring Agent

Claude reasons over the tools on a schedule and decides — alert, or stay quiet. No alert fatigue.

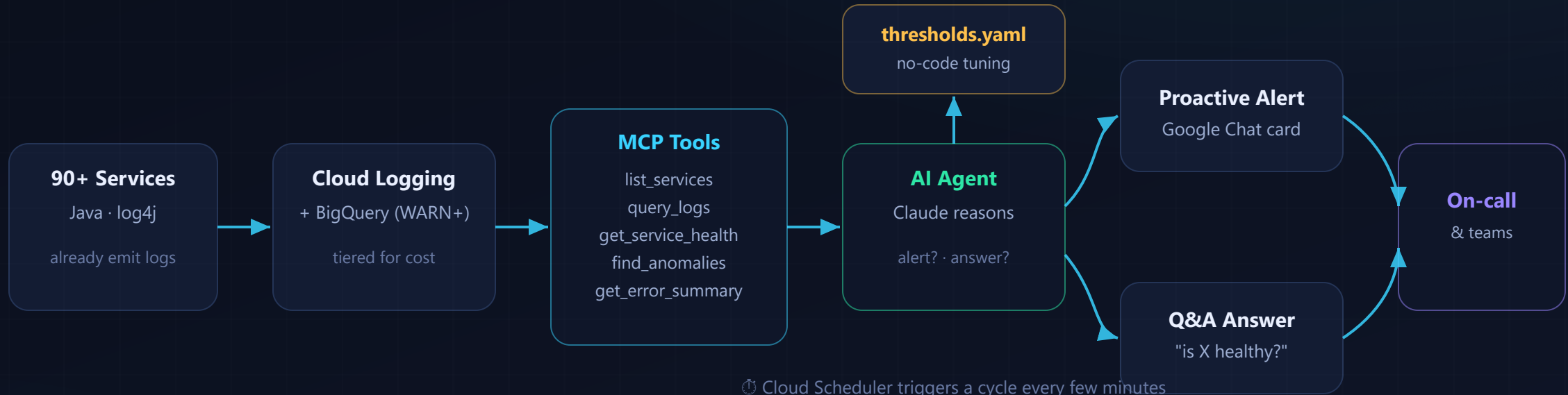


### Google Chat Integration

Pushes proactive alerts and answers support questions interactively, right where teams already work.

● HOW IT WORKS · OVERALL FLOW

# From a log line to a decision — end to end.



# Five tools. Zero ability to change anything.

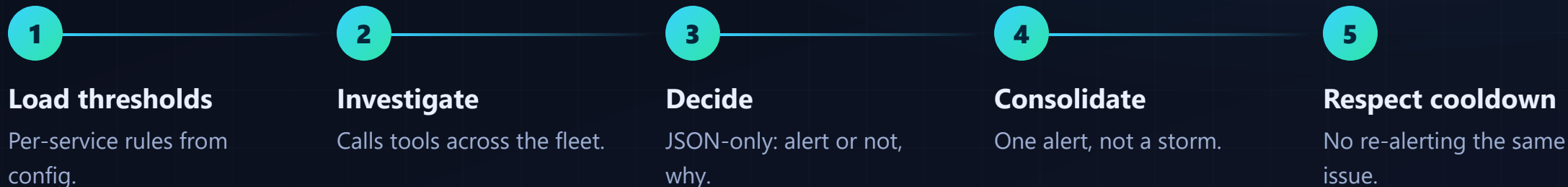
| TOOL                            | WHAT IT ANSWERS                                       | GUARDRAIL                            |
|---------------------------------|-------------------------------------------------------|--------------------------------------|
| <code>list_services</code>      | Which services are emitting logs right now?           | read-only                            |
| <code>query_logs</code>         | Show me recent logs (service, severity, window, text) | capped $\leq 500$ rows · $\leq 24$ h |
| <code>get_service_health</code> | Error %, fatals, p99 latency, status for a service    | derived, not stored                  |
| <code>find_anomalies</code>     | What's breaching thresholds across the fleet?         | sorted by severity                   |
| <code>get_error_summary</code>  | Top error signatures, similar errors clustered        | normalized grouping                  |

 No mutation handlers anywhere

 Same tools power both the agent and the chat bot

# Smart enough to stay quiet.

Every cycle, the agent investigates with the tools and returns a single, structured decision — engineered against alert fatigue.



```
// the agent must return only this – no guesswork, no invented numbers
{ "shouldAlert": true, "anomalies": [ ... ], "summary": "payment-service: 1
FATAL in last 5m" }
```

# Alerts that explain themselves. Answers on demand.

## LogSentry Alert — 1 anomaly

PAYMENT-SERVICE · FATAL

1 FATAL log (> threshold 0) — service may be degraded

**Observed 1** vs **threshold 0** · severity 200

OutOfMemoryError: Java heap space

[View in Cloud Logging →](#)

## LogSentry

### QUESTION

is payment-service healthy?

Yes — over the last 5 minutes: 0.4% error rate, p99 latency 180 ms, no fatals. Status: **healthy**.

Sources: get\_service\_health, find\_anomalies



# Change sensitivity without shipping code.

```
# config/thresholds.yaml
defaults:
  error_rate: { max_error_pct: 5.0, min_volume: 50 }
  fatal: { max_fatal: 1 }
  latency: { p99_ms_threshold: 2000 }

services:
  payment-service: # stricter for money paths
    error_rate: { max_error_pct: 1.0 }
    fatal: { max_fatal: 0 }
  batch-report-service: # quiet by design
    silence: { window_minutes: 360 }
```

- Global defaults, with **field-level per-service overrides**.
- Stricter rules for critical paths; looser for batch jobs.
- Validated on load — a typo fails fast with a clear error.
- Edit, redeploy, done. **No engineer required.**

- CUSTOMER SAFETY FIRST

# It **cannot** hurt production. By construction.

- **Read-only everywhere.** No tool, query, or script can write to any production system.
- **Least-privilege identity.** The runtime account holds only three viewer roles — no editor, no owner.
- **Query caps.** Hard limits on rows and time windows bound both cost and blast radius.
- **SQL guard.** Any mutating keyword is rejected before a query runs.

## Runtime permissions — the entire list

roles/logging.viewer

roles/bigquery.dataViewer

roles/bigquery.jobUser

That's it. The agent **literally cannot** change production.

- BUILT TO LOWER THE BILL

# The biggest cost lever, pulled by default.

- **Log tiering.** Only `severity ≥ WARNING` is exported to BigQuery; high-volume INFO/DEBUG stay in the cheaper default bucket.
- **Query caps** bound every scan — no runaway BigQuery bills.
- **Scale-to-zero.** Cloud Run costs nothing when idle between cycles.
- **Alert dedup** stops alert storms that burn API calls and attention.

## Where the savings come from

~80%

of log volume is INFO/DEBUG — kept out of the expensive path\*

\*Typical fleet ratio; actuals depend on app verbosity. Sampling at the log4j appender can cut it further.

# Serverless, standard, and boring — on purpose.



## Runtime

TypeScript on Node 20, deployed to **Cloud Run**. Cloud Scheduler is the heartbeat.



## Integration

Official **MCP SDK**, Google Cloud Logging + BigQuery, Anthropic Claude, Google Chat.



## Design

Every external call is behind an **injectable interface** — fully testable offline.

No new agents on services

Uses logging we already pay for

Deploys via 5 idempotent scripts

• ENGINEERING RIGOR

# Shipped with proof, not promises.

80

Automated tests, all passing

>85%

Coverage on core logic (100% on detection & tools)

100%

Offline — tests need zero cloud credentials

- Built in **8 gated phases** — no phase advanced until its tests passed.
- Unit + integration coverage of detectors, tools, MCP server, chat bot, and the agent cycle.

## Verified behaviors

Alert-once-then-cooldown · malformed AI output never crashes or false-alarms · invalid input returns a clean error · unauthorized chat calls rejected.

## • THE PLAN

# A safe, staged rollout.

1

### Shadow

Deploy read-only. Dry-run cycles — observe decisions, post nothing.

2

### Pilot

Enable alerts for 2–3 critical services in a non-prod Chat space.

3

### Tune

Teams calibrate thresholds. Measure signal-to-noise.

4

### Fleet

Roll to all 90+ services. Enable interactive Q&A org-wide.

### Low risk

Read-only from day one. Worst case: it stays silent.

### Fast value

Q&A is useful on day one, before any tuning.

### Reversible

One config flag returns to dry-run instantly.

- WHY IT MATTERS

# What changes for the business.

- **Detect before customers do** — shift incidents left, protect revenue and trust.
- **Minutes, not hours** — anyone answers "is X healthy?" without a senior engineer.
- **Less on-call toil** — consolidated, deduped alerts mean fewer 3 a.m. pages.
- **Lower, predictable log spend** — tiering + caps bound the bill as we grow.

## Targets to measure

↓ **MTTD**

Mean time to *detect* — the metric LogSentry moves first

↓ **Toil**

Manual log-triage hours per incident

# Greenlight the shadow + pilot rollout on our critical services.

Low risk, read-only, reversible — and useful from day one. LogSentry is built, tested, and ready to deploy.

Built & tested

Read-only & safe

Ready to deploy



**LogSentry**

LogSentry · Nirav Mehta

Engineered by **Nirav Mehta** · Questions & discussion